Manual técnico y de la aplicación web

Referencia para entender el código, ejecutarlo y mantenerlo. Complementa `ARQUITECTURA.md` y `ESQUEMA_BD.md`.

1. Tecnologías y versiones

Capa	Tecnología
Frontend	React 18, Vite 5, React Router, CSS3
Backend	Node.js 20, Express 5
Base de datos	PostgreSQL 16, Sequelize 6
Autenticación	JWT (`jwt-simple`), bcryptjs, `google-auth-library` , reCAPTCHA v2
Imágenes	Sharp (transformaciones), face-api.js + canvas (rostros)
Almacenamiento	Amazon S3 (`@aws-sdk/client-s3`) o disco local
Despliegue	AWS EC2, nginx, PM2, Let's Encrypt

2. Estructura del proyecto

```
Controlador/   user.js, publicacion.js, follow.js, imagen.js  (lógica de negocio)
rutas/         rutas.js                                        (endpoints REST)
modelo/        user, publicacion, follower, imageVariant, faceDetection (+ index.js)
servicios/     jwt.js, storage.js, imageProcessor.js
middlewares/   auth.js                                         (valida el JWT)
database/      conexion.js, initSchema.js, schema.sql
models/        pesos de face-api.js (no son modelos de datos)
frontend/src/  App.jsx, api.js, auth.jsx, config.js, components/, pages/
deploy/        bootstrap, nginx, políticas IAM/S3, respaldo
docs/          documentación
```

3. Aplicación web (frontend)

SPA en React con enrutamiento del lado del cliente. Toda la comunicación con el servidor pasa por `src/api.js`, que añade el encabezado `Authorization` con el token guardado.

Página	Función
`Inicio`	Bienvenida y acceso a iniciar sesión o registrarse.
`Login`	Correo + contraseña + reCAPTCHA, o botón de Google.
`Registro`	Alta de cuenta con foto de perfil opcional y reCAPTCHA.
`FeedVista`	Galería de miniaturas de todas las publicaciones.
`Siguiendo`	Publicaciones de las personas que sigues.
`Perfil`	Datos del usuario y sus fotos; crear, editar y borrar.
`EditarPerfil`	Actualizar datos y foto de perfil.

Componentes principales: `Galeria` (cuadrícula de miniaturas), `Lightbox` (vista ampliada con tira de versiones, 10 filtros y análisis de rostros), `GoogleButton`, `Recaptcha`, `PostCard`, `Navbar` (barra superior en escritorio, inferior en móvil).

Configuración (`src/config.js`): `VITE_API_URL`, `VITE_GOOGLE_CLIENT_ID`, `VITE_RECAPTCHA_SITE_KEY`. En producción se compila con `VITE_API_URL=/api`, así el navegador llama al mismo dominio y nginx reenvía a Node.

4. API REST

Prefijo `/api`. Las rutas marcadas con 🔒 exigen el encabezado `Authorization: <token JWT>`.

Método	Ruta	Descripción
POST	`/registrar/user`	Crea una cuenta (valida reCAPTCHA, guarda hash bcrypt).
POST	`/login/user`	Inicio de sesión con correo y contraseña; devuelve el token.
POST	`/login/google`	Valida el token de Google en el servidor y devuelve el token propio.
GET 🔒	`/profile/user/:id`	Datos de un usuario.
GET 🔒	`/listar/user/:page`	Lista paginada de usuarios.
PUT 🔒	`/actualizar/user`	Actualiza datos del usuario.
PUT 🔒	`/upload/user`	Sube la foto de perfil.
POST 🔒	`/crear/publicacion`	Crea una publicación.
POST 🔒	`/subir-imagen/:id`	Sube la foto de una publicación; genera transformaciones y analiza rostros.
GET 🔒	`/listar/publicacion`	Feed global.
GET 🔒	`/publicacion/:id`	Una publicación con sus versiones.
PUT 🔒	`/actualizar/publicacion/:id`	Edita el texto.
DELETE 🔒	`/borrar/publicacion/:id`	Borra la publicación.
PUT 🔒	`/likes/publicacion`	Da o quita "me gusta".
POST 🔒	`/follow/seguir/:id`	Seguir a un usuario.
DELETE 🔒	`/follow/dejar/:id`	Dejar de seguir.
GET 🔒	`/follow/comprobar/:id`	¿Ya lo sigo?
GET 🔒	`/listar/siguiendo`	Publicaciones de quienes sigo.
GET 🔒	`/follow/usuario-nick/:nick`	Buscar usuario por nick.
POST 🔒	`/imagen/transformar`	Aplica un filtro manual (grayscale, sepia, invert, blur, bright, contrast, warm, cool, hue) y lo guarda.
POST 🔒	`/analizar-rostro`	Analiza rostros de una foto bajo demanda.

5. Procesamiento de imágenes

Al subir una foto (`servicios/imageProcessor.js`):

1. Se guarda el **original** en el almacenamiento.
2. Con **Sharp** se generan tres transformaciones: **miniatura** (reducción de escala), **blanco y negro** y **sepia**.
3. Se registran cuatro filas en `image_variants` (clave de almacenamiento, no el binario).

4. **face-api.js** detecta rostros, puntos faciales, edad y género; el resultado va a `face_detections`.

Las claves quedan bajo `post/` (publicaciones) y `usuarios/` (perfiles). La base de datos guarda solo la referencia a la clave.

6. Almacenamiento (`servicios/storage.js`)

Interfaz común con dos implementaciones, elegidas con `STORAGE_DRIVER`:

Valor	Destino	Uso
`local`	Carpeta `imagenes/`	Desarrollo, sin cuenta de AWS.
`s3`	Bucket de S3	Producción. Usa la cadena de credenciales por defecto del SDK: en la EC2 toma el rol IAM de la instancia.

7. Variables de entorno

Variable	Descripción
`PORT`	Puerto de Node (4000).
`NODE_ENV`	`production` en el servidor.
`BASE_URL`, `CORS_ORIGIN`	URL pública y origen permitido por CORS.
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SSL`	Conexión a PostgreSQL.
`JWT_SECRET`	Clave de firma de las sesiones.
`GOOGLE_CLIENT_ID`	Client ID para validar el token de Google.
`RECAPTCHA_SITE_KEY`, `RECAPTCHA_SECRET_KEY`	Claves de reCAPTCHA v2.
`STORAGE_DRIVER`, `AWS_REGION`, `AWS_S3_BUCKET`	Almacenamiento. En EC2 no se definen claves de acceso (rol IAM).
`REKOGNITION_ENABLED`	Activa Amazon Rekognition (opcional, desactivado).

El archivo `.env` nunca se sube al repositorio (`.gitignore`); `.env.example` es la plantilla.

8. Ejecución en desarrollo

```
npm install
cp .env.example .env     # completar valores
npm run db:init          # crea las tablas
npm run dev              # API en http://localhost:4000

cd frontend
npm install
npm run dev              # interfaz en http://localhost:5173
```

9. Operación en producción

Tarea	Comando
Estado de la aplicación	`pm2 status`
Ver registros	`pm2 logs mi-red-social`
Reiniciar	`pm2 restart mi-red-social`
Recompilar el frontend	`cd frontend && VITE_API_URL=/api npm run build`
Probar y recargar nginx	`sudo nginx -t && sudo systemctl reload nginx`
Estado del certificado	`sudo certbot certificates`
Respaldo de la BD	`deploy/backup-db.sh`

PM2 está configurado para arrancar con el sistema; si la instancia se reinicia, la aplicación se levanta sola.

9.1 Convenciones de código

- Código dividido por capas (rutas → controladores → modelos/servicios) y comentado en español.
- Los controladores responden JSON con `status` y `mensaje`.
- Los secretos solo viven en variables de entorno.
- Ramas de GitHub por integrante para identificar quién desarrolló cada parte.

10. Problemas conocidos

- El análisis facial usa el backend JavaScript de TensorFlow (sin aceleración nativa), por lo que tarda varios segundos en la t3.micro.
- Las imágenes del bucket son de lectura pública para poder mostrarse en el feed; los datos personales no se guardan en las imágenes.
- El Login con Google exige un origen HTTPS autorizado en el Client ID del equipo.